

T0.4 — INBOUND_WEBHOOK_SECRET sin fallback

Diagnostico de solo lectura · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Diferencia clave con T0.2 y T0.3: hay tokens emitidos vivos en servicios externos (Zapier/Make/n8n). Antes de implementar T0.4, hay que decidir el plan de rotacion (seccion 12) segun el escenario actual de Render.
Necesito confirmacion del owner sobre si INBOUND_WEBHOOK_SECRET ya esta seteado y si hay tokens vivos.

1. Donde se usa INBOUND_WEBHOOK_SECRET

Un solo modulo lo lee: `agent/inbound_tokens.py:32` dentro de la funcion `_secreto()`. Esa funcion la consumen dos rutas criticas:

Funcion	Llamada desde	Que hace
<code>_secreto() -> generar_token(telefono)</code>	<code>agent/main.py:526-527 (GET /admin/inbound/token)</code>	Emite tokens HMAC para que el owner los pegue en Zapier/Make/n8n.
<code>_secreto() -> verificar_token(token)</code>	<code>agent/main.py:550-551 (POST /webhook/inbound/{token})</code>	Valida el token recibido del servicio externo y devuelve el telefono asociado.

INBOUND_WEBHOOK_SECRET aparece tambien documentada en `.env.example:145-151` como variable opcional.

2. Fallback actual si la variable no esta configurada

`agent/inbound_tokens.py:30-42`:

```
def _secreto() -> bytes:
    secret = os.getenv("INBOUND_WEBHOOK_SECRET", "").strip()
    if secret:
        return secret.encode("utf-8")
    # Fallback en dev: derivar de ADMIN_TOKEN si existe, sino un valor fijo de dev.
    admin = os.getenv("ADMIN_TOKEN", "").strip()
    if admin:
        logger.warning("[INBOUND] INBOUND_WEBHOOK_SECRET no configurado, derivando de ADMIN_TOKEN")
        return hashlib.sha256(b"inbound-webhook-derived|" + admin.encode("utf-8")).digest()
    logger.warning("[INBOUND] INBOUND_WEBHOOK_SECRET no configurado – usando valor de desarrollo INSEGURO")
    return b"dona-inbound-dev-secret-do-not-use-in-prod"
```

Tres caminos en orden de preferencia:

- **Secret real** (INBOUND_WEBHOOK_SECRET no vacio) — ruta segura.
- **Derivado de ADMIN_TOKEN**: SHA256(b'inbound-webhook-derived|' + ADMIN_TOKEN). Warning. Inseguro en produccion.
- **Literal hardcodeado** b'dona-inbound-dev-secret-do-not-use-in-prod'. Warning. Catastrofico si se llega aqui.

3. Uso de ADMIN_TOKEN, valores derivados o literales de desarrollo

Path	Inseguridad	Por que
Derivar de ADMIN_TOKEN	Alta en produccion	El derivado es sha256(prefijo_publico + ADMIN_TOKEN). El prefijo esta en el repo publico ('inbound-webhook-derived'). Cualquiera con acceso a ADMIN_TOKEN (un dev, un leak de logs, un dump de Render env) puede recomputar el secret y forjar tokens. Acopla dos secretos que deberian ser independientes.
Literal 'dona-inbound-dev-secret-do-not-use-in-prod'	Critica en produccion	El literal esta en el repo publico de GitHub. Cualquier persona en Internet puede usarlo si Dona corre con ese fallback. Forja tokens para cualquier telefono que el atacante invente.

4. Endpoints / integraciones que dependen del secret

4.1 GET /admin/inbound/token?telefono= (agent/main.py:513-537)

- **Acceso:** solo owner (Bearer ADMIN_TOKEN).
- **Output:** token + URL <BASE_URL>/webhook/inbound/<token> lista para pegar en Zapier/Make/n8n.
- **Quien lo usa:** el owner. Cero impacto si la URL del endpoint no cambia.

4.2 POST /webhook/inbound/{token} (agent/main.py:540-569)

- **Acceso:** publico (no requiere admin token).
- **Body:** {'mensaje': 'texto...'}
- **Comportamiento:** si el token es valido, envia el mensaje via proveedor.enviar_mensaje (Meta o Whapi). Trunca a 4000 chars.
- **Quien lo usa:** servicios externos (Zapier/Make/n8n triggers, Shopify, Stripe events del owner, etc.).

4.3 Integraciones reales conectadas

No hay registro en codigo de que Zaps/Makes estan activos. El owner es el unico que sabe. **Cada token emitido vive donde el owner lo pego.**

5. Tokens ya emitidos que podrian invalidarse con una rotacion

Realidad tecnica: cada token esta firmado con el secret vigente al momento de emision. Si el secret cambia, **todos** los tokens previos quedan invalidos (responden 403 al primer POST).

Necesito tu confirmacion operativa sobre el estado actual de la variable en Render — esto define el alcance del impacto:

Escenario	Efecto del cambio T0.4 + setear INBOUND_WEBHOOK_SECRET
A. INBOUND_WEBHOOK_SECRET ya esta en Render con un valor real	Cero impacto. El cambio T0.4 solo cierra paths fallback. Tokens existentes siguen validos.
B. INBOUND_WEBHOOK_SECRET no esta, ADMIN_TOKEN si (path derivado activo hoy)	Tokens existentes firmados con derivado. Setear INBOUND_WEBHOOK_SECRET nuevo = invalidacion masiva .
C. Ninguna de las dos (literal activo)	Tokens existentes firmados con el literal publico. Compromiso preexistente — hay que rotar si o si.

6. Riesgo real actual si el fallback sigue activo en produccion

6.1 Escenario A (INBOUND_WEBHOOK_SECRET ya seteado)

Riesgo bajo. El check de import ni siquiera se ejecuta porque la variable esta. T0.4 solo blinda codigo.

6.2 Escenario B (derivado de ADMIN_TOKEN)

Riesgo **alto**:

- Cualquier filtracion de ADMIN_TOKEN (logs, dump, ex-empleado) entrega tambien el secret de inbound webhooks.
- Atacante recomputa el HMAC, forja un token valido para <cualquier-numero>, y hace POST /webhook/inbound/<forjado> con {'mensaje': 'Spam!'}
- Dona envia spam por Meta/Whapi al telefono que el atacante elija. **Costos, abuso del proveedor** (puede llevar a baneo de Meta App o WHAPI_TOKEN), **violacion de TCPA**.

6.3 Escenario C (literal hardcodeado)

Riesgo **catastrofico**:

- El secret esta en GitHub publico. Cualquiera puede armar tokens validos.
- Mismo abuso que arriba pero sin barrera de entrada.
- Si Dona corre asi en produccion aunque sea unas horas, se debe asumir **compromiso de tokens** y rotar de inmediato.

6.4 Defensa actual

agent/main.py:540-569 valida el token con compare_digest (timing-safe, bien) pero **no hay**:

- Rate limiting en /webhook/inbound/{token} (un atacante puede martillar el endpoint).
- Logging detallado de intentos rechazados (los 403 quedan en log estandar).

- TTL de tokens (los tokens no expiran 'por diseno' segun el docstring).

T0.4 no resuelve eso. Cierra solo el agujero del secret. El rate limiting y el TTL son tareas separadas.

7. Variables que deben estar en Render antes del merge

Solo una:

Variable	Origen	Formato	Comando para generar
INBOUND_WEBHOOK_SEC RET	string aleatorio largo, decision del owner	URL-safe, 32+ chars	python -c "import secrets; print(secrets.token_urlsafe(32))"

No solicito ver el valor. Solo confirmacion de que esta seteada en Render con un valor que **no es derivado de ADMIN_TOKEN ni el literal del repo.**

ADMIN_TOKEN no se modifica en este PR. Sigue siendo el Bearer del endpoint /admin/*.

8. Cambio minimo recomendado

Patron identico al de T0.2 (Stripe). El modulo `inbound_tokens.py` se importa al startup desde `main.py`, asi que un check al import-time aborta el deploy si falta el secret.

8.1 agent/inbound_tokens.py

Agregar al **top del modulo** (tras el logger):

```
_ENVIRONMENT = os.getenv("ENVIRONMENT", "development").lower()

if _ENVIRONMENT == "production" and not os.getenv("INBOUND_WEBHOOK_SECRET", "").strip():
    raise RuntimeError(
        "[INBOUND] INBOUND_WEBHOOK_SECRET no configurado en produccion - "
        "los tokens caerian a un fallback derivado de ADMIN_TOKEN o a un "
        "literal publico del repo, lo que permitiria a un atacante forjar "
        "tokens para enviar mensajes WhatsApp a cualquier numero. "
        "Configura la variable antes de reintentar el deploy."
    )
```

Modificar `_secreto()` para defensa en profundidad:

```
def _secreto() -> bytes:
    secret = os.getenv("INBOUND_WEBHOOK_SECRET", "").strip()
    if secret:
        return secret.encode("utf-8")
    # En produccion no debemos llegar aqui (el check de import lo evita), pero
    # como defensa en profundidad rechazamos via RuntimeError si la env var
    # desaparece despues del startup.
    environment = os.getenv("ENVIRONMENT", "development").lower()
    if environment == "production":
        raise RuntimeError(
            "[INBOUND] INBOUND_WEBHOOK_SECRET no configurado en produccion"
        )
    # Fallback dev/test: derivar de ADMIN_TOKEN o literal. Solo no-prod.
    admin = os.getenv("ADMIN_TOKEN", "").strip()
    if admin:
        logger.warning("[INBOUND] ... derivando de ADMIN_TOKEN (INSEGURO, solo dev/test)")
        return hashlib.sha256(b"inbound-webhook-derived|" + admin.encode("utf-8")).digest()
    logger.warning("[INBOUND] ... usando valor de desarrollo INSEGURO (solo dev/test)")
    return b"dona-inbound-dev-secret-do-not-use-in-prod"
```

8.2 agent/main.py

Agregar `import agent.inbound_tokens` # *noqa: F401* despues del import de `agent.billing` que ya esta. Una linea. Fuerza el check al startup.

8.3 .env.example

Reformular el bloque actual (linea 145-151) para enfatizar obligatoriedad en produccion. Sin valores reales.

8.4 Total de cambio

- `agent/inbound_tokens.py`: +12 lineas top-level + ~6 lineas en `_secreto()`.
- `agent/main.py`: +1 linea.
- `.env.example`: ajustar bloque existente, +3 lineas.
- `tests/test_inbound_tokens.py`: 4 tests nuevos.
- **PR pequeno, ~25-30 lineas netas.**

9. Archivos que tocaria

Archivo	Tipo de cambio
agent/inbound_tokens.py	Top-level: <code>_ENVIRONMENT</code> + check <code>RuntimeError</code> si production sin secret. Modificar <code>_secreto()</code> para rechazar en production (defensa en profundidad).
agent/main.py	Agregar <code>'import agent.inbound_tokens # noqa: F401'</code> despues del import de <code>agent.billing</code> .
.env.example	Reformular bloque <code>INBOUND_WEBHOOK_SECRET</code> enfatizando obligatoriedad en produccion. Sin valores reales.
tests/test_inbound_tokens.py	Agregar <code>TestSecretoProduction</code> con 4 tests.

Lo que NO toco:

- `agent/main.py:webhook_inbound` (handler) — su logica ya rechaza con 403 si `verificar_token` retorna `None`. No requiere cambios.
- `agent/main.py:admin_generar_token_inbound` — sin cambios.
- Comportamiento de `generar_token` y `verificar_token` (la logica HMAC) — sin cambios. Solo cambia de donde sale el secret.
- Rate limiting, TTL, audit log de inbound — fuera del alcance de T0.4 (tareas separadas).

10. Tests a correr

10.1 Tests existentes a actualizar

- 8 tests actuales pasan tal cual con el monkeypatch de `INBOUND_WEBHOOK_SECRET='test-secret-xyz'` (autouse fixture en linea 12-15). **Sin cambios necesarios.**
- El fixture autouse ya garantiza que el secret esta seteado, asi que el check de production no levanta en entorno de tests.

10.2 Tests nuevos a agregar

```
class TestSecretoProduction:
    """T0.4: el modulo no debe arrancar en produccion sin INBOUND_WEBHOOK_SECRET."""

    def test_production_sin_secret_levanta_runtime_error_al_reload(self, monkeypatch):
        import importlib
        monkeypatch.setenv("ENVIRONMENT", "production")
        monkeypatch.delenv("INBOUND_WEBHOOK_SECRET", raising=False)
        try:
            with pytest.raises(RuntimeError, match="INBOUND_WEBHOOK_SECRET"):
                importlib.reload(inbound_tokens)
        finally:
            monkeypatch.setenv("ENVIRONMENT", "test")
            monkeypatch.setenv("INBOUND_WEBHOOK_SECRET", "test-secret-xyz")
            importlib.reload(inbound_tokens)

    def test_production_secret_solo_whitespace_levanta(self, monkeypatch):
        # Similar: con valor " " levanta RuntimeError.

    def test_production_runtime_sin_secret_levanta_en_secreto(self, monkeypatch):
        # Defensa en profundidad: si el secret desaparece despues del import.

    def test_production_con_secret_funciona(self, monkeypatch):
```

```
# Con secret valido, _secreto() retorna los bytes correctos.
```

Total: 8 existentes + 4 nuevos = **12 tests** en test_inbound_tokens.py.

10.3 Suite completa

`pytest -q`. Estimado: 455 actuales + 4 nuevos = **~459 tests**, ~92 s.

11. Riesgos de compatibilidad y deploy

Riesgo	Probabilidad	Mitigacion
Render sin INBOUND_WEBHOOK_SECRET -> deploy aborta	Solo si escenario era B o C antes del merge	Setear primero, mergear despues
Tokens emitidos antes del PR son invalidados por la rotacion	Solo si escenario B o C -> necesita re-emision	Plan de rotacion de §12
ADMIN_TOKEN se sigue derivando en dev	OK por diseno (path dev permitido)	Solo cambia en produccion
Tests existentes rompen	Minima — el fixture autouse ya setea el secret	Plan: ningun test existente rompe
Reload del modulo en tests deja estado roto	Baja, ya manejado en T0.2	Patron try/finally con restore

Compatibilidad con Zapier/Make/n8n:

- El formato del token NO cambia.
- El endpoint `/webhook/inbound/{token}` sigue identico.
- Solo cambia el secret bajo el cual se firma. Si el secret en Render es el mismo que ya estaba activo -> cero impacto.

12. Plan de rotacion seguro

Esta es la **parte distinta** de T0.4 vs T0.2 y T0.3. Hay tokens emitidos vivos en servicios externos.

12.1 Antes de empezar el PR — Diagnostico operativo

El owner debe responder (yo no necesito ver valores):

- **Esta hoy INBOUND_WEBHOOK_SECRET seteado en Render?**
- Si si -> escenario A. Cero rotacion. PR seguro.
- Si no:
 - **Se esta usando hoy /webhook/inbound/?** Que Zaps/Makes/n8ns estan conectados? Hay tokens ya emitidos en uso real?
 - Si **no hay tokens en uso** -> escenario 'facil': setear el secret, mergear, listo. No hay impacto.
 - Si **hay tokens en uso** -> escenario 'rotacion coordinada' (§12.2 abajo).

12.2 Rotacion coordinada (solo si escenario B/C con tokens vivos)

Pasos en orden estricto:

- **Inventario** (offline, owner): listar todos los Zaps/Makes/n8ns que apuntan a /webhook/inbound/... Anotar el telefono asociado a cada uno (no el token, no el secret — solo el telefono).
- **Generar nuevo secret aleatorio:** `python -c "import secrets; print(secrets.token_urlsafe(32))"`. El owner copia el valor (yo no lo veo).
- **Setear INBOUND_WEBHOOK_SECRET en Render** con ese valor. Render redeploya (despliegue de la env var sin codigo nuevo). En ese momento: tokens viejos quedan invalidos (responden 403). Cero deploy de codigo nuevo todavia.
- **Verificar en Render dashboard** que el deploy con la nueva env termino OK.
- **Mergear el PR T0.4.** Render redeploya con el codigo nuevo. El check del import pasa porque el secret esta.
- **Re-emitir tokens** para cada telefono del inventario: GET /admin/inbound/token?telefono=<E.164>. El owner pega cada nueva URL en el Zap/Make/n8n correspondiente.
- **Probar** un disparo desde cada Zap -> confirmar que llega el mensaje a WhatsApp.

Tiempo estimado de la ventana de rotura: entre paso 3 y paso 7. Durante ese tiempo, los Zaps externos disparan POST que devuelven 403. El owner debe coordinar con sus integraciones para minimizar esa ventana.

12.3 Variante 'rotacion zero-downtime' (no recomendada para este PR)

Soportar **dos secrets simultaneamente** (INBOUND_WEBHOOK_SECRET_PRIMARY + INBOUND_WEBHOOK_SECRET_PREVIOUS) durante una ventana de transicion. verificar_token aceptaria tokens firmados por cualquiera. generar_token solo usa el primary.

- **Pros:** los tokens viejos siguen funcionando hasta que se haya migrado el ultimo Zap.
- **Contras:** ~30 lineas extra de codigo + mas complejidad. Si Dona tiene pocas integraciones (probable), no vale la pena.
- **Recomendacion:** no implementar en T0.4. Si el owner reporta que tiene 10+ Zaps activos, lo podemos retomar como T0.4-bis.

13. Checklist antes de mergear

13.1 Pre-PR (antes de empezar)

- ☐ Confirmar el estado actual de `INBOUND_WEBHOOK_SECRET` en Render (escenario A/B/C).
- ☐ Si escenario B o C: hacer inventario de Zaps/Makes/n8ns activos antes de tocar nada.
- ☐ Decidir nombre de branch: sugiero **pr/inbound-webhook-secret-obligatorio**.
- ☐ Si hay tokens vivos: avisar a los servicios externos del corte planificado.

13.2 Pre-merge (durante rotacion si aplica)

- ☐ Generar nuevo `INBOUND_WEBHOOK_SECRET` con `secrets.token_urlsafe(32)`.
- ☐ Setear en Render (sin mergear todavia).
- ☐ Verificar que el deploy con la nueva env termino OK.
- ☐ (Opcional) Re-emitir tokens de prueba y validar que disparan mensajes.

13.3 Durante el PR (local)

- ☐ Implementar cambio en `agent/inbound_tokens.py` (top-level + `_secreto()`).
- ☐ Agregar import en `agent/main.py`.
- ☐ Reformular `.env.example`.
- ☐ Agregar `TestSecretoProduction` (4 tests).
- ☐ `pytest -q` localmente — 100% verde.

13.4 Pre-push

- ☐ Diff toca solo los 4 archivos previstos.
- ☐ Ningun secret real en codigo, comentarios ni tests.
- ☐ Commit: *fix(seguridad): INBOUND_WEBHOOK_SECRET obligatorio en produccion*.

13.5 Post-push, pre-merge

- ☐ Push a origin/pr/inbound-webhook-secret-obligatorio.
- ☐ Confirmar visualmente que `INBOUND_WEBHOOK_SECRET` esta en Render.
- ☐ Si hay rotacion pendiente: completarla antes de mergear.

13.6 Post-merge

- ☐ Render deploy debe completar sin `RuntimeError`.
- ☐ Probar GET `/admin/inbound/token` con un telefono -> URL valida.
- ☐ Probar POST `/webhook/inbound/<token>` con la URL recién emitida -> mensaje llega a WhatsApp.
- ☐ Si hay Zaps existentes que aun no se actualizaron: completar la migracion.
- ☐ Verificar logs por 403 inesperados (Zaps todavia con token viejo).

14. Resumen ejecutivo

Aspecto	Detalle
Riesgo que cierra	P0 — token forjable que permite enviar mensajes WhatsApp a cualquier numero (escenarios B y C). En escenario A solo blinda codigo.
Patron	Identico a T0.2: check al import del modulo + defensa en profundidad en <code>_secreto()</code> .
Archivos	4 (1 codigo, 1 import 1 linea, 1 docs, 1 tests)
Tamano	~25-30 lineas netas
Tests	0 actualizados + 4 nuevos
Env var	INBOUND_WEBHOOK_SECRET obligatoria en Render
Compatibilidad	Tokens existentes solo siguen validos si el secret no cambia. Si hay rotacion -> seguir §12.2
Riesgo de deploy	Bajo si la matriz env vars esta consistente
Reversible	Si, redeploy del commit anterior
Dependencia operativa	Inventario de Zaps/Makes/n8ns antes de la rotacion si aplica

Estado: no implementado todavia. Esperando OK del owner con confirmacion de:

- Estado actual de INBOUND_WEBHOOK_SECRET en Render (escenario A/B/C).
- Si hay tokens emitidos vivos en Zapier/Make/n8n y cuantos aproximadamente.
- Si autorizas la rotacion coordinada o preferis 'no hay tokens vivos, mergear directo'.